

AI Game Move Optimization

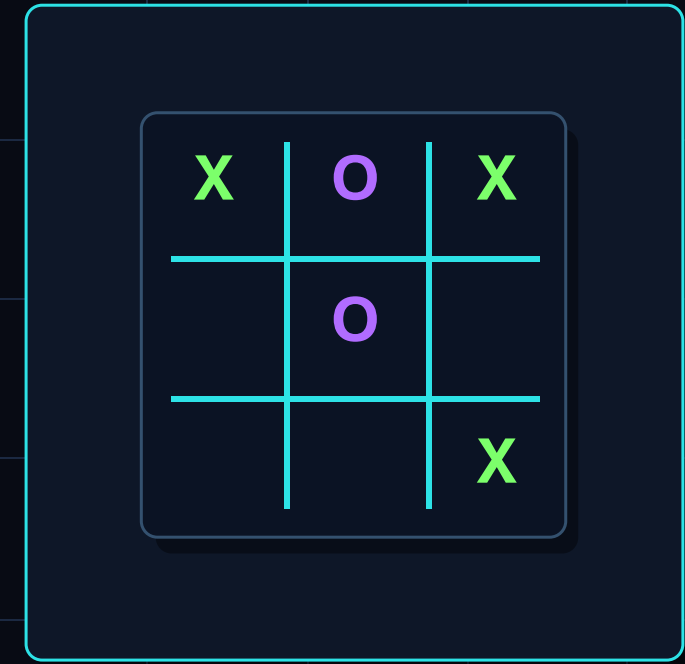
Using algorithms to predict smart moves in Tic Tac Toe

Minimax

Alpha-Beta Pruning

Search Tree Optimization

Core idea: evaluate future game states and select the move that maximizes winning chance while limiting the opponent.



AI

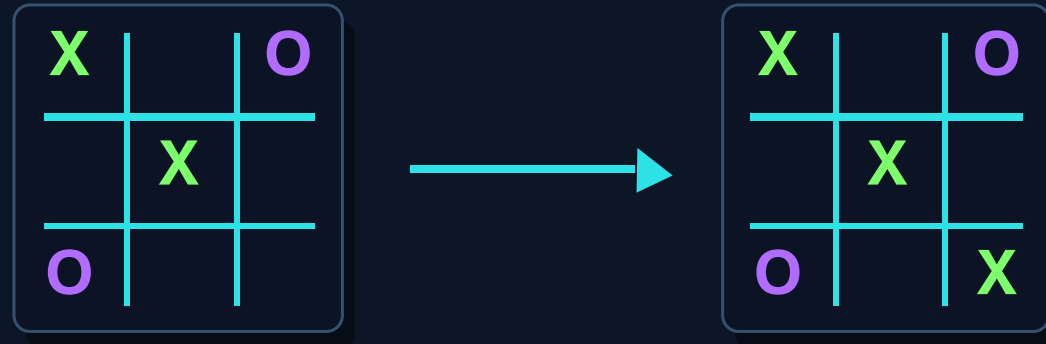
Move

Introduction to Game AI

Game AI is decision-making logic that chooses actions based on goals, opponent behavior, and future outcomes.

How Game AI Thinks

- Represents the game as states
- Generates all legal actions
- Predicts opponent responses
- Scores outcomes using utility values



Decision = choose the move with best future value

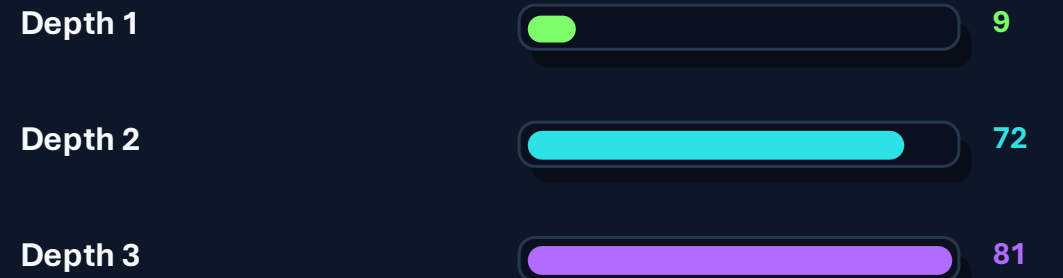
Problem Statement

How can an AI choose the best move efficiently when the number of possible futures grows quickly?

Challenge

- A move creates many new game states
- Every opponent response adds more branches
- Full search can become expensive
- Optimization is needed before the AI can scale

State-Space Growth



Repeated game-state evaluation motivates pruning and smarter search.

Minimax Algorithm

Minimax explores possible moves recursively and assumes both players make optimal decisions.

Working Principle

MAX player maximizes score.

MIN player minimizes score.

The root move with the best backed-up value is selected.

Win = +1

Draw = 0

Loss = -1



Values are propagated upward from terminal states.

Alpha-Beta Pruning

Alpha-Beta gives the same optimal result as Minimax, but avoids evaluating branches that cannot affect the final decision.

Before Pruning



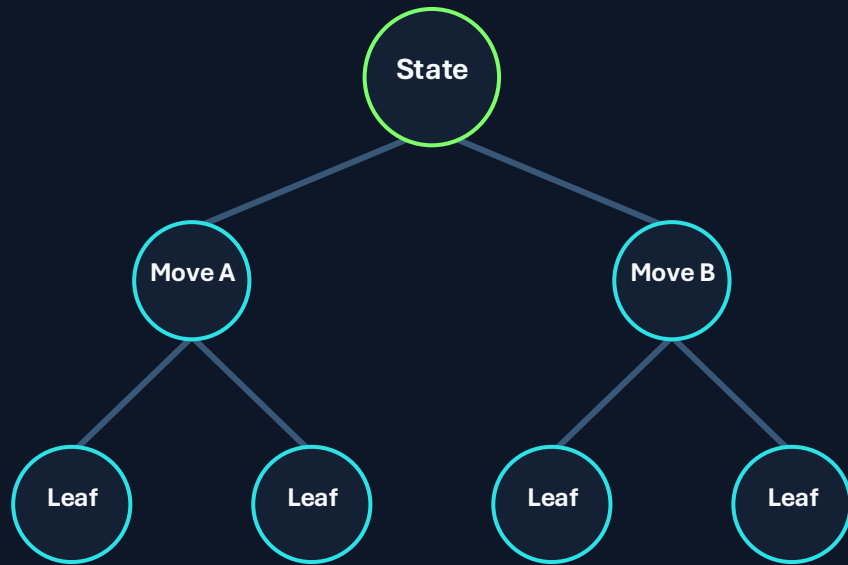
After Pruning



Pruned branch

Search Tree & ADA Concepts

The project maps directly to tree traversal, recursion, backtracking, and complexity analysis.



ADA Mapping

Branching factor (b)

Depth (d)

Recursion

Backtracking

DFS-style traversal

Game moves become child nodes; traversal returns the best utility.

Complexity Analysis

Optimization matters because game-tree search grows exponentially with branching factor and depth.

Minimax

Time Complexity

$$O(b^d)$$

Explores every legal path up to depth d .

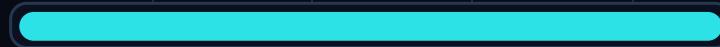
Alpha-Beta

Best Case Complexity

$$O(b^{(d/2)})$$

Skips branches that cannot change the final move.

Minimax nodes



81

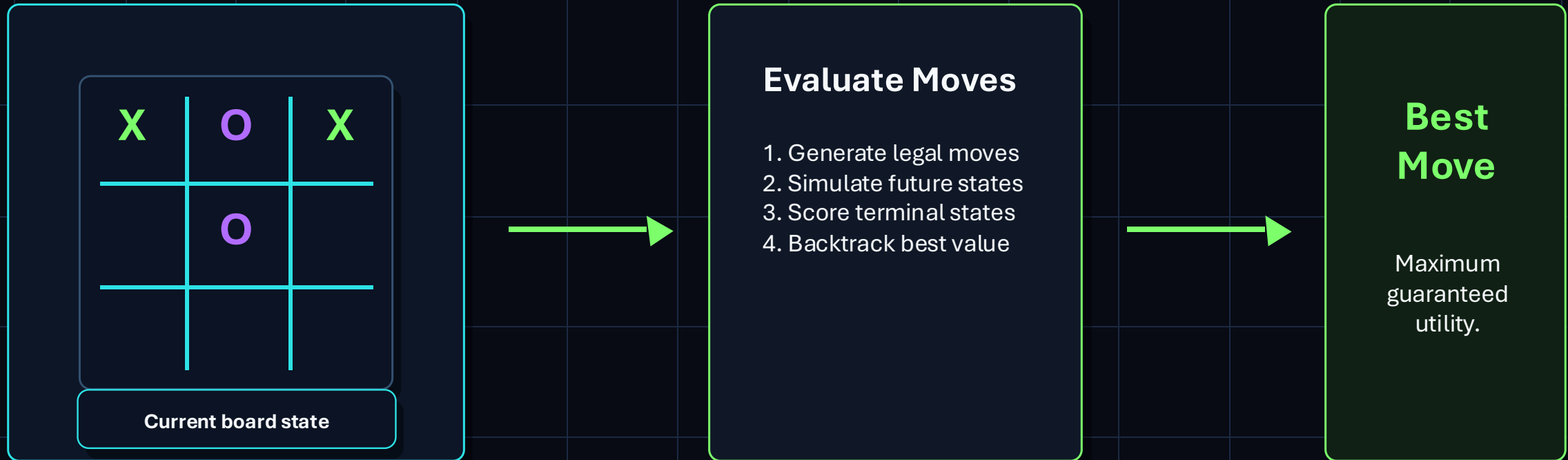
Alpha-Beta nodes



27

Implementation: Tic Tac Toe AI

The implementation evaluates game states and returns the optimal next move for the AI player.

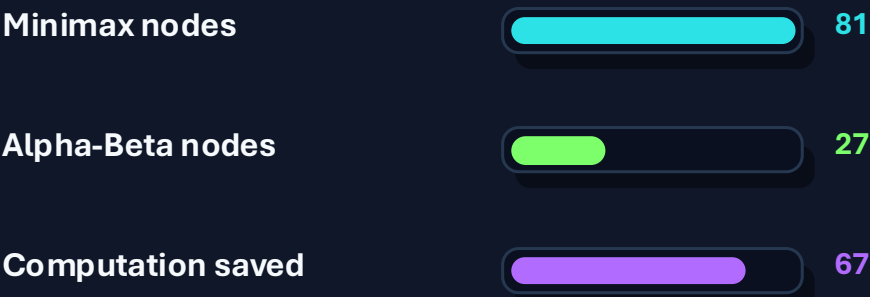


Results & Algorithm Comparison

Both algorithms find the optimal move, but Alpha-Beta reduces unnecessary computation.

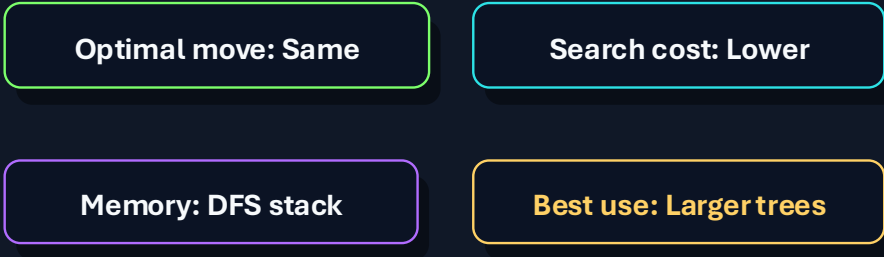


Performance View



Illustrative Tic Tac Toe comparison: pruning avoids branches that cannot influence the final result.

Comparison Summary



Result: Alpha-Beta is preferred when the same answer is needed with fewer evaluations.

Conclusion

AI game move optimization demonstrates how ADA concepts power intelligent decision systems.



Key Takeaways

- Games can be modeled as search trees
- Minimax provides optimal decision logic
- Alpha-Beta reduces node exploration
- Complexity analysis explains performance gain

Final Statement

Optimization is essential in AI systems. Alpha-Beta Pruning significantly improves Minimax while preserving the same best move decision.

ADA concepts applied: recursion, trees, pruning, complexity